EKSAMENSOPPGAVE

Emnekode:	**DAT107**
Emnenavn:	**Databaser**
Dato:	**04.01.2024**

Eksamensform:	Skriftlig digital
Eksamenstid:	09:00 – 13:00
Antall eksamensoppgaver:	4
Antall sider (inkludert denne):	7
Antall vedlegg:	0
Tillatte hjelpemidler:	Ingen
Emneansvarlig:	Dag Toppe Larsen
Kommer til eksamenslokalet:	Hvis mulig

Telefonnummer emneansvarlig kan nås på under eksamen:

Dag Toppe Larsen:	483 07 293
Lars-Petter Helland:	928 28 046

Merknader:

Jeg bekrefter at arbeidet er selvstendig utarbeidet, og at jeg kjenner til hva *Forskrift om studium og eksamen ved Høgskulen på Vestlandet, sier om fusk og forsøk på fusk i § 12-1.*

Oppgave 1 (20%~48 minutter) – ORM/JPA

Vi skal jobbe litt med en database med oversikt over **film**er og **person**er med tilknytning til filmene. I første omgang tar vi bare med hvem som er **regissør**.

En film må ha **én og bare én** regissør. En person kan har regissert **ingen, én eller flere** filmer. Vi har altså et en-til-mange forhold.

Vi tenker oss følgende tabeller. Det er tatt med eksempeldata for å illustrere mulig innhold.

film

id (PK) **Surrogatnøkkel**	tittel	aar	sjanger	regissor
1	'The Dark Knight'	2008	'Superhelt'	1
2	'Harry Potter and …'	2001	'Fantasy'	3
3				
…	…	…	…	…

person

id (PK) **Surrogatnøkkel**	fornavn	etternavn	fodselsaar	nasjonalitet
1	'Christopher'	'Nolan'	1970	'USA'
2	'Christian'	'Bale'	1974	'England'
3	'Chris'	'Columbus'	1958	'USA'
4	'Daniel Jacob'	'Radcliffe'	1989	'England'
5				
…	…	…	…	…

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (8% ~ 18 min) Skriv Java-klassene for entitetstypene som trengs for å representere eksempelet vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer.
   Vi ønsker å tilrettelegge for **toveis navigering** i Java mellom film og person, slik vi lett kan se hvilke filmer en person har regissert.

Du kan i de videre oppgavene anta at entitetsklassene inneholder de nødvendige konstruktører, gettere og settere, etc. du trenger i løsningene dine.

Vi antar at vi har en hjelpeklasse **FilmDAO**. Du skal lage et par metoder i denne. Du kan anta at en EntityManagerFactory kalt emf er opprettet og tilgjengelig.

b) (4% ~ 10 min) Skriv en metode **`hentFilmerRegissertAv(int personId)`** i FilmDAO som henter ut en liste av filmer en person har regissert. Hvis ingen person med denne id-en finnes eller vedkommende ikke har regissert noen filmer, skal det i begge tilfeller returneres en tom liste.

c) (4% ~ 10 min) Skriv en metode **`hentFilmerISjanger(String sjanger)`** i FilmDAO som henter ut en liste av filmer i en gitt sjanger. Hvis ingen film i denne sjangeren finnes, skal det returneres en tom liste.

Som en siste oppgave skal vi se på skuespillere i filmer. En **person** kan være **skuespiller** (spille en rolle) i en **film**. Dette er et naturlig mange-til-mange forhold, siden en film kan ha flere skuespillere og en person kan ha spilt i flere filmer. Det kan f.eks. modelleres slik.

skuespiller

filmid (PK, FK) **Del av sammensatt PK**	personid (PK, FK) **Del av sammensatt PK**	rolle
1	2	'Batman'
2	4	'Harry Potter'
…	…	…

og i Java blir det slik:

```
@Entity
@IdClass(SkuespillerPK.class)
public class Skuespiller {

      @Id @ManyToOne @JoinColumn(name="filmid")
      private Film film;
      @Id @ManyToOne @JoinColumn(name="personid")
      private Person person;
      private String rolle;

      // + konstruktører, gettere og settere
}

public class SkuespillerPK {
      private Integer film;
      private Integer person;
}
```

d) (4% ~ 10 min) Skriv en metode **`leggInnSkuespiller(Integer filmId, Integer personId, String rolle)`** i FilmDAO som registrerer en person som skuespiller i en gitt rolle i en film. Du kan anta at både aktuell film og person allerede finnes i databasen, og at personen ikke er registrert som skuespiller i denne filmen tidligere.

Oppgave 2 (40%~96 minutter) – Modellering/normalisering

Problembeskrivelse

En høgskole har flere campus. Hver campus har navn (for eksempel Kronstad), telefonnr, samt postadresse (gate, husnummer, postnr, poststed).

Høgskolen har også flere fakultet. Et fakultet kan lokalisert på en eller flere campus. Det kan også være flere fakultet lokalisert på samme campus. Et fakultet har navn og leder (dekan) som er en ansatt.

Det er mange ansatte på høgskolen. Hver ansatt har navn, fødselsdato, ansattnr, fødselsnr og kjønn. En ansatt har en bestemt campus som arbeidsted og arbeider bare på et fakultet.

Oppgave

- Bestem hva som er hensiktsmessige tabeller for denne databasen.
- Lag (tegn) en logisk (fullstendig) ER-modell for denne databasen i en av notasjonene som har vært brukt i kurset.
- Vis at tabellene for databasen minst tilfredsstiller 3. normalform.

Oppgaven kan bli besvart helhetlig (istedenfor punktvis) i den grad du mener dette er enklere. Men alle disse punktene skal besvares.

Huskeliste for hva besvarelsen skal (minst) inneholde:

- Primærnøkler (som du mener er mest hensiktsmessig)
- Fremmednøkler
- Datatyper
- Min/maks kardinalitet
- Sterke/svake entitetstyper og eksistensavhengighet/uavhengighet (kråkefot-notasjon) eller type aggregering (UML-notasjon)
- Redegjørelse/forklaring på at denne tilfredsstiller hver av 1., 2., og 3. normalform.
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekt på både praktisk utførelse og teoretisk forståelse**
- **Besvarelsen skal være "konsistent", dvs. skal være for den "samme" databasen, og de valgene man har tatt i en del av besvarelsen skal ikke "motsies" av de valgene man har tatt i en annen del av besvarelsen.**
- (Dersom du har problemer med deler av oppgaven, prøv å gjøre det du kan. Man kan for eksempel buke både modellering og normalisering som uavhengige fremgangsmåter for å bestemme tabeller for databasen.)

Oppgave 3 (20%~48 minutter) – SQL

Problembeskrivelse

Denne oppgaven er løst basert på problembeskrivelsen i Oppgave 2 – Modellering/normalisering. Du kan anta at en mulig, "redusert" løsning inneholder tabellene:

- Ansatt(Ansattnr, navn, campus*)
- Campus(Navn, telefonnr)

Disse tabellene skal være utgangspunkt for besvarelsene i denne oppgaven. Du kan derfor løse denne oppgaven selv om du ikke fikk til Oppgave 2.

Oppgave

- Skriv fullstendig SQL-kode for å opprette tabellen Ansatt. Du kan anta at tabellen Campus eksisterer.
- Skriv SQL-kode for å legge inn en ansatt (med selvvalgte data)
- Skriv SQL-spørring for å hente ut all info om alle ansatte – inkludert telefonnr til Campus
- Anta at det er mulig for en ansatt å ikke ha en campus (NULL-merke). Skriv SQL-spørring for å hente ut all info om ansatte – inkludert telefonnr til Campus. Ansatt uten campus skal ikke tas med.
- Skriv SQL-spørringen ovenfor som relasjonsalgebra.
- Skriv en SQL-spørring som finner antall ansatte på hver campus.
- Skriv en SQL-spørring som skriver ut ansatte sortert på navn
- Skriv SQL-kode for å opprette en indeks på kolonnen Navn i tabellen Ansatt.

Oppgaven kan bli besvart helhetlig (istedenfor punktvis) i den grad du mener dette er enklere. Men alle disse punktene skal besvares.

Huskeliste for hva besvarelsen skal (minst) inneholde:

- Primærnøkler (som du mener er mest hensiktsmessig)
- Fremmednøkler
- Datatyper
- Null/not null
- Eventuelle fremmednøkler og on update/delete krav.
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekt på både praktisk utførelse og teoretisk forståelse**
- **Besvarelsen skal være "konsistent", dvs. skal være for den "samme" databasen, og de valgene man har tatt i en del av besvarelsen skal ikke "motsies" av de valgene man har tatt i en annen del av besvarelsen.**
- (Dersom du har problemer med deler av oppgaven, prøv å gjøre det du kan.)

Oppgave 4 (20%~48 minutter) – NoSQL

Problembeskrivelse

Ta utgangspunkt problembeskrivelsen i Oppgave 2 – Modellering/normalisering. Bare selve problembeskrivelsen i Oppgave 2 – ikke løsningen du kom frem til der. Du kan derfor løse denne oppgaven selv om du ikke fikk til Oppgave 2.

Oppgave

- Lag en dokument-database (data blir gjentatt – ikke normalisert) som kan tilfredsstille problembeskrivelsen. Man kan anta at NoSQL-funksjonalitet i PostgreSQL blir brukt.
- Bestemme de ulike typer dokument databasen trenger (en dokument-"type" kan minne om en tabell i relasjonsdatabaser)
- Et eksempel på et JSON eller XML-dokument (med data) for hver av disse dokument-typene
- Anta at databasen er implementert i PostgreSQL som tabeller med en kolonne ID og en kolonne DATA som inneholder selve dokumentene. Vis et eksempel på å legge inn et dokument i databasen (med data i JSON eller XML)
- Forklar hva som er fordeler og ulemper med dokumentdatabaser i forhold til tradisjonelle relasjonsdatabaser. Hva kan gjøre at man velger å bruke enten det ene eller det andre? Hva mener du passer best til å løse denne problembeskrivelsen, og hvorfor?

Oppgaven kan bli besvart helhetlig (istedenfor punktvis) i den grad du mener dette er enklere. Men alle disse punktene skal besvares.

Huskeliste for hva besvarelsen skal (minst) inneholde:

- Basert på enten JSON eller XML-dokument
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekk på både praktisk utførelse og teoretisk forståelse**
- **Besvarelsen skal være "konsistent", dvs. skal være for den "samme" databasen, og de valgene man har tatt i en del av besvarelsen skal ikke "motsies" av de valgene man har tatt i en annen del av besvarelsen.**
- (Dersom du har problemer med deler av oppgaven, prøv å gjøre det du kan.)